Abstract Data Types: Linked Lists in C

Implementation, Operations, and 60+ Practice Problems

Juevesano, D. — University of the Philippines Cebu
Course Material Supplement — Data Structures and Algorithms

1 Abstract Data Type — Overview

Definition 1.1 (Abstract Data Type)**.** An **Abstract Data Type (ADT)** is a mathematical model consisting of a set of values and a set of operations defined over those values, without specifying how those values are stored or how the operations are implemented. The client interacts solely through the defined interface.

Remark 1.1. This document covers two *linked* realisations of the List ADT.

Part I — Singly-Linked List (SLL): each node stores one data value and a single `next` pointer. A `head` pointer references the first node and a `tail` pointer references the last node, enabling $O(1)$ prepend and $O(1)$ append. Traversal is unidirectional (front to back only).

Part II — Doubly-Linked List (DLL): each node stores one data value, a `next` pointer, and a `prev` pointer. `head` and `tail` sentinels bound the list. Every insertion and deletion at either end costs $O(1)$; arbitrary-position operations cost $O(n)$ due to traversal. Bidirectional traversal is possible.

The client-facing interface is identical to the ArrayList and VectorList interfaces defined in earlier documents. The *representation* changes; the *contract* does not. No `const`, no `static`, and no global variables are used anywhere.

Part I: Singly-Linked List (SLL)

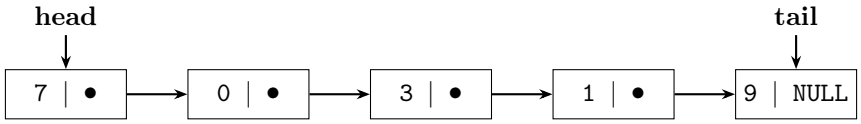
2 SLL — Interface Contract

Operation	Signature (C)	Semantics
`init`	`void init(SLL *L)`	Initialise empty list.
`push_back`	`int push_back(SLL *L, int x)`	Append x at the rear via tail.
`push_front`	`int push_front(SLL *L, int x)`	Prepend x at the front via head.
`insert`	`int insert(SLL *L, int x, int pos)`	Insert x before index `pos`.
`pop_back`	`int pop_back(SLL *L, int *out)`	Remove last element into `*out`.
`pop_front`	`int pop_front(SLL *L, int *out)`	Remove first element into `*out`.
`removeAt`	`int removeAt(SLL *L, int pos)`	Remove element at index `pos`.
`getItem`	`int getItem(SLL L, int pos)`	Return element at index `pos`.
`setItem`	`int setItem(SLL *L, int x, int pos)`	Overwrite element at index `pos`.
`size`	`int size(SLL L)`	Return number of elements.
`isEmpty`	`int isEmpty(SLL L)`	Return 1 if empty, 0 otherwise.
`isFull`	`int isFull(SLL L)`	Always returns 0 (unbounded).
`freeList`	`void freeList(SLL *L)`	Free all nodes; reset to empty.

Table 1: SLL ADT interface contract.

3 SLL — Implementation

3.1 Memory Layout



size = 5

3.2 Struct Definition

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct SLLNode {
    int data;               /* payload                          */
    struct SLLNode *next;   /* pointer to the next node         */
} SLLNode;

typedef struct {
    SLLNode *head;          /* pointer to the first node        */
    SLLNode *tail;          /* pointer to the last node         */
    int     sz;             /* number of valid elements         */
} SLL;
```

Listing 1: SLL node and list type definitions.

3.3 Internal Node Allocator

```c
/*
 * newNode : allocate one SLLNode on the heap.
 * Returns a pointer to the new node, or NULL on allocation failure.
 */
SLLNode *newNode(int data) {
    SLLNode *n = (SLLNode *) malloc(sizeof(SLLNode));
    if (n == NULL)
        return NULL;
    n->data = data;
    n->next = NULL;
    return n;
}
```

Listing 2: newNode — allocate and initialise a single node.

3.4 Complete Implementation

```c
/* init : set head, tail to NULL and sz to 0. */
void init(SLL *L) {
    L->head = NULL;
    L->tail = NULL;
    L->sz   = 0;
}

/* size : return the number of valid elements. */
int size(SLL L) {
```

```c
    return L.sz;
}

/* isEmpty : return 1 if the list holds no elements, 0 otherwise. */
int isEmpty(SLL L) {
    return (L.sz == 0);
}

/*
 * isFull : a heap-based list is never logically full.
 * Always returns 0 (bounded only by available heap memory).
 */
int isFull(SLL L) {
    (void) L;
    return 0;
}
```

Listing 3: SLL — init, size, isEmpty, isFull.

```c
/*
 * getItem : return the element at position pos (0-indexed).
 * Returns -1 if pos is out of bounds.
 * Time : O(n) -- must traverse to reach pos.
 */
int getItem(SLL L, int pos) {
    SLLNode *cur;
    int i;

    if (pos < 0 || pos >= L.sz)
        return -1;
    cur = L.head;
    for (i = 0; i < pos; i++)
        cur = cur->next;
    return cur->data;
}

/*
 * setItem : overwrite the element at position pos with value x.
 * Returns 1 on success, 0 if pos is out of bounds.
 * Time : O(n).
 */
int setItem(SLL *L, int x, int pos) {
    SLLNode *cur;
    int i;

    if (pos < 0 || pos >= L->sz)
        return 0;
    cur = L->head;
    for (i = 0; i < pos; i++)
        cur = cur->next;
    cur->data = x;
    return 1;
}
```

Listing 4: SLL — getItem, setItem.

```c
/*
 * push_back (append) :
```

```
 * Allocate a new node for x and link it after tail.
 * Update tail; if the list was empty, also update head.
 * Returns 1 on success, 0 on allocation failure.
 *
 * Before : head -> <a0, ..., a_{n-1}> <- tail   size = n
 * After  : head -> <a0, ..., a_{n-1}, x> <- tail  size = n+1
 *
 * Time : O(1) -- direct tail access.
 */
int push_back(SLL *L, int x) {
    SLLNode *n = newNode(x);
    if (n == NULL)
        return 0;
    if (L->tail == NULL) {      /* list was empty */
        L->head = n;
        L->tail = n;
    } else {
        L->tail->next = n;
        L->tail = n;
    }
    L->sz++;
    return 1;
}

/*
 * push_front (prepend) :
 * Allocate a new node for x and link it before head.
 * Update head; if the list was empty, also update tail.
 * Returns 1 on success, 0 on allocation failure.
 *
 * Before : head -> <a0, a1, ..., a_{n-1}> <- tail
 * After  : head -> <x, a0, a1, ..., a_{n-1}> <- tail
 *
 * Time : O(1).
 */
int push_front(SLL *L, int x) {
    SLLNode *n = newNode(x);
    if (n == NULL)
        return 0;
    if (L->head == NULL) {      /* list was empty */
        L->head = n;
        L->tail = n;
    } else {
        n->next  = L->head;
        L->head  = n;
    }
    L->sz++;
    return 1;
}
```

Listing 5: SLL — push_back, push_front.

```
/*
 * insert :
 * Insert x before position pos.
 * Special cases:
 *   pos == 0       -> equivalent to push_front
 *   pos == size(L) -> equivalent to push_back
```

```
 * Returns 1 on success, 0 on invalid pos or allocation failure.
 *
 * Time : O(n) -- traversal to pos-1.
 */
int insert(SLL *L, int x, int pos) {
    SLLNode *prev, *n;
    int i;

    if (pos < 0 || pos > L->sz)
        return 0;
    if (pos == 0)
        return push_front(L, x);
    if (pos == L->sz)
        return push_back(L, x);

    n = newNode(x);
    if (n == NULL)
        return 0;

    prev = L->head;
    for (i = 0; i < pos - 1; i++)
        prev = prev->next;

    n->next    = prev->next;
    prev->next = n;
    L->sz++;
    return 1;
}
```

Listing 6: SLL — insert.

```
/*
 * pop_front :
 * Remove the first node; store its data in *out.
 * Update head (and tail if list becomes empty).
 * Returns 1 on success, 0 if empty.
 * Time : O(1).
 */
int pop_front(SLL *L, int *out) {
    SLLNode *old;
    if (L->head == NULL)
        return 0;
    old     = L->head;
    *out    = old->data;
    L->head = old->next;
    if (L->head == NULL)        /* list is now empty */
        L->tail = NULL;
    free(old);
    L->sz--;
    return 1;
}

/*
 * pop_back :
 * Remove the last node; store its data in *out.
 * Must traverse to the second-to-last node to update tail.
 * Returns 1 on success, 0 if empty.
 * Time : O(n) -- no prev pointer available.
```

```c
 */
int pop_back(SLL *L, int *out) {
    SLLNode *cur, *prev;
    if (L->head == NULL)
        return 0;
    *out = L->tail->data;
    if (L->head == L->tail) {   /* single-element list */
        free(L->head);
        L->head = NULL;
        L->tail = NULL;
        L->sz--;
        return 1;
    }
    /* Traverse to the second-to-last node */
    prev = NULL;
    cur  = L->head;
    while (cur->next != NULL) {
        prev = cur;
        cur  = cur->next;
    }
    prev->next = NULL;
    L->tail    = prev;
    free(cur);
    L->sz--;
    return 1;
}

/*
 * removeAt :
 * Remove the element at position pos.
 * Returns 1 on success, 0 if pos is out of bounds.
 * Time : O(n).
 */
int removeAt(SLL *L, int pos) {
    SLLNode *prev, *target;
    int i, dummy;

    if (pos < 0 || pos >= L->sz)
        return 0;
    if (pos == 0)
        return pop_front(L, &dummy);

    prev = L->head;
    for (i = 0; i < pos - 1; i++)
        prev = prev->next;

    target     = prev->next;
    prev->next = target->next;
    if (target->next == NULL)   /* removed the tail */
        L->tail = prev;
    free(target);
    L->sz--;
    return 1;
}
```

Listing 7: SLL — pop_front, pop_back, removeAt.

```c
/*
```

```c
 * freeList :
 * Traverse the list and free every node, then reset fields.
 * MUST be called when the SLL is no longer needed.
 */
void freeList(SLL *L) {
    SLLNode *cur, *nxt;
    cur = L->head;
    while (cur != NULL) {
        nxt = cur->next;
        free(cur);
        cur = nxt;
    }
    L->head = NULL;
    L->tail = NULL;
    L->sz   = 0;
}

/* print (utility -- not part of the ADT interface)
 * Display the list in the form <a0, a1, ..., a_{n-1}>
 */
void print(SLL L) {
    SLLNode *cur;
    int i;
    printf("List (size=%d): <", L.sz);
    cur = L.head;
    i   = 0;
    while (cur != NULL) {
        printf("%d", cur->data);
        if (cur->next != NULL) printf(", ");
        cur = cur->next;
        i++;
    }
    printf(">\n");
}
```

Listing 8: SLL — freeList and print utility.

3.5 Operation Complexity Summary — SLL

Operation	Time	Note
`init`, `freeList`	$O(1)$ / $O(n)$	freeList traverses all nodes
`push_back`	$O(1)$	Direct tail access
`push_front`	$O(1)$	Direct head access
`pop_front`	$O(1)$	Direct head access
`pop_back`	$O(n)$	No prev pointer; must traverse
`insert(pos)`	$O(n)$	Traverse to pos-1
`removeAt(pos)`	$O(n)$	Traverse to pos-1
`getItem`, `setItem`	$O(n)$	Sequential traversal
`size`, `isEmpty`, `isFull`	$O(1)$	Field reads

Table 2: SLL time complexity summary.

4 Global Rules for All Practice Problems

1. Use *only* the ADT operations: `getItem`, `setItem`, `insert`, `removeAt`, `push_back`, `push_front`, `pop_back`, `pop_front`, `size`, `isEmpty`, `isFull`, `init`, `freeList`. Direct pointer manipulation is *not* permitted unless a problem explicitly states otherwise.
2. No global variables, no `const` qualifiers, no `static` qualifiers.
3. Pass parameters by pointer wherever mutation is required; pass by value otherwise.
4. Declare all variables at the top of each function (C89 style).
5. Return types follow the implementation: `int` for operations that can fail, `void` for those that cannot.
6. Only `stdio.h` and `stdlib.h` are permitted unless a problem states otherwise.
7. Always call `freeList` on every SLL or DLL when it is no longer needed. Memory leaks are bugs.

5 SLL — Practice Problems

Problems are curated from or inspired by MIT 6.006, CMU 15-122, Stanford CS106B, Princeton COS 226, UC Berkeley CS61B, Sedgewick & Wayne *Algorithms* (4th ed.), and LeetCode, adapted to the SLL ADT interface.

Problem 1 — Reverse an SLL In-Place

Statement. Given an SLL L of n integers, reverse it in-place so that the element originally at index i ends up at index $n-1-i$.

Example. $\langle 1,2,3,4,5\rangle \Rightarrow \langle 5,4,3,2,1\rangle$.

Constraints. $O(n)$ time, $O(1)$ space. No auxiliary list.

Source: MIT 6.006 Recitation 2; LeetCode #206 (Reverse Linked List).

```
/*
 * reverse : reverse L in-place by re-wiring next pointers.
 *
 * Strategy (iterative, three-pointer):
 *   prev = NULL, cur = head.
 *   At each step: save cur->next, point cur->next back to prev,
 *   advance prev and cur.
 *   After the loop, head = prev (new front), tail = old head.
 *
 * Time : O(n) -- single pass.
 * Space : O(1) -- three pointer variables.
 *
 * NOTE: This problem directly manipulates node pointers because
 *       reversing a linked list requires re-wiring -- it cannot
 *       be expressed purely through the value-level ADT ops
 *       without O(n) extra space.  The instructor may allow this
 *       exception; otherwise use the value-swap approach below.
 */
void reverse(SLL *L) {
    SLLNode *prev, *cur, *nxt;

    if (L->sz <= 1)
        return;

    L->tail = L->head;   /* old head becomes new tail */

```

```
    prev = NULL;
    cur  = L->head;
    while (cur != NULL) {
        nxt        = cur->next;
        cur->next  = prev;
        prev       = cur;
        cur        = nxt;
    }
    L->head = prev;
}
```

Listing 9: Problem 1 — Reverse SLL in-place (pointer re-wiring).

```
/*
 * reverseValues : swap values between symmetric positions.
 * Uses only getItem / setItem / size -- pure ADT approach.
 *
 * Time : O(n^2) -- each getItem/setItem is O(n) for an SLL.
 * Space : O(1).
 */
void reverseValues(SLL *L) {
    int lo, hi, tmp;

    lo = 0;
    hi = size(*L) - 1;
    while (lo < hi) {
        tmp = getItem(*L, lo);
        setItem(L, getItem(*L, hi), lo);
        setItem(L, tmp, hi);
        lo++;
        hi--;
    }
}
```

Listing 10: Problem 1 (alternative) — Reverse using value swaps via ADT.

Problem 2 — Remove Duplicates from a Sorted SLL

Statement. Given a sorted SLL, remove all duplicate values so each value appears exactly once.

Example. $\langle 1, 1, 2, 3, 3, 3, 4, 5, 5 \rangle \Rightarrow \langle 1, 2, 3, 4, 5 \rangle$.

Constraints. $O(n)$ time, $O(1)$ auxiliary space.

Source: LeetCode #83 (Remove Duplicates from Sorted List); CMU 15-122.

```
/*
 * removeDuplicatesSorted : eliminate consecutive equal elements.
 *
 * Strategy (pointer walk):
 *   cur walks the list.  Whenever cur->data == cur->next->data,
 *   remove cur->next in-place (save, re-wire, free).
 *   Otherwise advance cur.
 *
 * Time : O(n) -- single pass.
 * Space : O(1).
 */
void removeDuplicatesSorted(SLL *L) {
    SLLNode *cur, *dup;
```

```
14
15      cur = L->head;
16      while (cur != NULL && cur->next != NULL) {
17          if (cur->data == cur->next->data) {
18              dup         = cur->next;
19              cur->next   = dup->next;
20              if (dup->next == NULL)   /* dup was the tail */
21                  L->tail = cur;
22              free(dup);
23              L->sz--;
24          } else {
25              cur = cur->next;
26          }
27      }
28  }
```

Listing 11: Problem 2 — Remove adjacent duplicates from sorted SLL.

Problem 3 — Move All Zeros to the End

Statement. Move all zero elements to the end of L while preserving the relative order of non-zero elements.

Example. $\langle 0, 1, 0, 3, 12 \rangle \Rightarrow \langle 1, 3, 12, 0, 0 \rangle$.

Constraints. $O(n)$ time, $O(1)$ auxiliary space. Use only ADT operations.

Source: LeetCode #283 (Move Zeroes); CMU 15-122 homework-style.

```
1   /*
2    * moveZeros : compact non-zero elements to front; fill rear with 0.
3    *
4    * Strategy (two-pass write-pointer -- value level):
5    *   Pass 1: for each i, if L[i] != 0 write it to position w, advance w
         .
6    *   Pass 2: fill positions [w .. n-1] with 0.
7    *
8    * Time : O(n^2) -- each setItem/getItem is O(n) for an SLL.
9    *                  O(n) pointer-level version shown below.
10   * Space : O(1).
11   */
12  void moveZeros(SLL *L) {
13      int w, i, n;
14
15      n = size(*L);
16      w = 0;
17
18      for (i = 0; i < n; i++) {
19          if (getItem(*L, i) != 0) {
20              setItem(L, getItem(*L, i), w);
21              w++;
22          }
23      }
24      for (i = w; i < n; i++)
25          setItem(L, 0, i);
26  }
```

Listing 12: Problem 3 — Move zeros to end (write-pointer via setItem).

Problem 4 — Detect a Cycle (Floyd's Algorithm)

Statement. Given an SLL (using raw pointers for this problem), return 1 if the list contains a cycle, 0 otherwise.

Constraints. $O(n)$ time, $O(1)$ space. This problem uses raw pointer access (not the value-level ADT) because cycle detection is a structural property.

Source: Floyd (1967); LeetCode #141 (Linked List Cycle); MIT 6.006 Lecture 9.

```
/*
 * hasCycle : return 1 if L contains a cycle, 0 otherwise.
 *
 * Strategy (Floyd's slow-fast pointers):
 *   slow advances one node per step; fast advances two.
 *   If they ever point to the same node, a cycle exists.
 *   If fast reaches NULL, the list is acyclic.
 *
 * Time : O(n).
 * Space : O(1).
 *
 * NOTE: Uses raw SLLNode pointers -- ADT ops are insufficient
 *       for structural cycle detection.
 */
int hasCycle(SLL L) {
    SLLNode *slow, *fast;

    slow = L.head;
    fast = L.head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return 1;
    }
    return 0;
}
```

Listing 13: Problem 4 — Floyd's tortoise-and-hare cycle detection.

Problem 5 — Find the Middle Node

Statement. Given an SLL with at least one node, return the *value* of the middle element. For even-length lists, return the value of the second middle.

Example. $\langle 1, 2, 3, 4, 5\rangle \Rightarrow 3$. $\langle 1, 2, 3, 4\rangle \Rightarrow 3$.

Source: LeetCode #876 (Middle of the Linked List); CMU 15-122.

```
/*
 * middleValue : return the value of the middle node.
 *
 * Strategy (slow-fast pointers):
 *   slow advances by 1; fast advances by 2.
 *   When fast reaches the end, slow is at the middle.
 *
 * Time : O(n).
 * Space : O(1).
```

```
 */
int middleValue(SLL L) {
    SLLNode *slow, *fast;

    slow = L.head;
    fast = L.head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}
```

Listing 14: Problem 5 — Middle node via slow-fast pointers.

Problem 6 — Merge Two Sorted SLLs

Statement. Given two sorted SLLs A and B, produce a new sorted SLL C containing all elements of A and B.

Example. $A = \langle 1, 3, 5 \rangle$, $B = \langle 2, 4, 6, 8 \rangle \Rightarrow C = \langle 1, 2, 3, 4, 5, 6, 8 \rangle$.

Source: Cormen et al. CLRS 4th ed., §2.3; LeetCode #21 (Merge Two Sorted Lists).

```
/*
 * mergeSorted : merge sorted A and B into new sorted list C.
 *
 * Strategy (two-pointer via ADT):
 *   ia, ib index into A and B.
 *   Always append the smaller head to C via push_back.
 *
 * Time : O((n+m)^2) via ADT (push_back is O(1) but getItem is O(n)).
 *        O(n+m) with raw pointer merge (shown below).
 * Space : O(n+m) for C.
 */
void mergeSorted(SLL A, SLL B, SLL *C) {
    int ia, ib;

    init(C);
    ia = 0;
    ib = 0;

    while (ia < size(A) && ib < size(B)) {
        if (getItem(A, ia) <= getItem(B, ib)) {
            push_back(C, getItem(A, ia));
            ia++;
        } else {
            push_back(C, getItem(B, ib));
            ib++;
        }
    }
    while (ia < size(A)) { push_back(C, getItem(A, ia)); ia++; }
    while (ib < size(B)) { push_back(C, getItem(B, ib)); ib++; }
}
```

Listing 15: Problem 6 — Merge two sorted SLLs into a third.

```c
/*
 * mergeSortedFast : O(n+m) pointer-level merge.
 *
 * Builds C by splicing nodes from A and B; A and B are
 * consumed (their nodes are reused -- do not call freeList on
 * A or B after this call).
 */
void mergeSortedFast(SLL *A, SLL *B, SLL *C) {
    SLLNode dummy, *tail_c, *a, *b;

    init(C);
    dummy.next = NULL;
    tail_c     = &dummy;

    a = A->head;
    b = B->head;

    while (a != NULL && b != NULL) {
        if (a->data <= b->data) {
            tail_c->next = a;
            a = a->next;
        } else {
            tail_c->next = b;
            b = b->next;
        }
        tail_c       = tail_c->next;
        tail_c->next = NULL;
        C->sz++;
    }
    while (a != NULL) {
        tail_c->next = a; a = a->next;
        tail_c = tail_c->next; tail_c->next = NULL; C->sz++;
    }
    while (b != NULL) {
        tail_c->next = b; b = b->next;
        tail_c = tail_c->next; tail_c->next = NULL; C->sz++;
    }
    C->head = dummy.next;
    C->tail = tail_c;
    A->head = A->tail = NULL; A->sz = 0;
    B->head = B->tail = NULL; B->sz = 0;
}
```

Listing 16: Problem 6 (fast) — O(n+m) pointer-level merge.

Problem 7 — Stack ADT Built on SLL

Statement. Implement a stack (push, pop, peek) using an SLL as the underlying storage. Push and pop at the *head* (front) for $O(1)$ amortised cost.

Source: CMU 15-122 Unit 2; Sedgewick & Wayne *Algorithms* 4th ed., §1.3.

```c
typedef SLL Stack;

void  sInit(Stack *S)            { init(S); }
int   sPush(Stack *S, int x)     { return push_front(S, x); }
int   sIsEmpty(Stack S)          { return isEmpty(S); }
void  sFree(Stack *S)            { freeList(S); }
```

```c

/*
 * sPop : remove and return the top element into *out.
 * Returns 1 on success, 0 on underflow.
 * Time : O(1) -- pop_front is O(1) for SLL.
 */
int sPop(Stack *S, int *out) {
    if (isEmpty(*S)) return 0;
    return pop_front(S, out);
}

/*
 * sPeek : return the top element without removing it.
 * Returns 1 on success, 0 if empty.
 * Time : O(1).
 */
int sPeek(Stack S, int *out) {
    if (isEmpty(S)) return 0;
    *out = S.head->data;   /* direct head access */
    return 1;
}
```

Listing 17: Problem 7 — Stack built on SLL (ADT layering).

Problem 8 — Queue ADT Built on SLL

Statement. Implement a FIFO queue using an SLL. Enqueue at the tail ($O(1)$); dequeue at the head ($O(1)$).

Source: CMU 15-122 Unit 2; MIT 6.006 Recitation; Sedgewick & Wayne §1.3.

```c
typedef SLL Queue;

void qInit(Queue *Q)  { init(Q); }
int  qIsEmpty(Queue Q){ return isEmpty(Q); }
void qFree(Queue *Q)  { freeList(Q); }

/*
 * enqueue : add x to the rear.  O(1) -- tail pointer.
 */
int enqueue(Queue *Q, int x) {
    return push_back(Q, x);
}

/*
 * dequeue : remove front element into *out.  O(1) -- head pointer.
 * Note: SLL achieves O(1) dequeue unlike ArrayList/VectorList.
 */
int dequeue(Queue *Q, int *out) {
    if (isEmpty(*Q)) return 0;
    return pop_front(Q, out);
}
```

Listing 18: Problem 8 — Queue built on SLL (O(1) both ends).

Problem 9 — Check if SLL is a Palindrome

Statement. Return 1 if L reads the same forwards and backwards, 0 otherwise.

Example. $\langle 1,2,3,2,1\rangle \Rightarrow 1$. $\langle 1,2,3\rangle \Rightarrow 0$.

Source: LeetCode #234 (Palindrome Linked List); Stanford CS106B.

```c
/*
 * isPalindrome : return 1 if L is a palindrome, 0 otherwise.
 *
 * Strategy: compare L[lo] and L[hi] moving inward.
 * Time : O(n^2) -- each getItem is O(n) for SLL.
 * Space : O(1).
 */
int isPalindrome(SLL L) {
    int lo, hi;

    lo = 0;
    hi = size(L) - 1;
    while (lo < hi) {
        if (getItem(L, lo) != getItem(L, hi))
            return 0;
        lo++;
        hi--;
    }
    return 1;
}
```

Listing 19: Problem 9 — Palindrome check via value comparison.

Problem 10 — Remove All Occurrences of a Value

Statement. Remove every occurrence of `val` from L in-place.

Example. $\langle 1,2,6,3,4,5,6\rangle$, `val`$=6 \Rightarrow \langle 1,2,3,4,5\rangle$.

Source: LeetCode #203 (Remove Linked List Elements); CMU 15-122.

```c
/*
 * removeValue : erase every node whose data == val.
 *
 * Strategy (sentinel / dummy-head technique):
 *   A dummy node before head simplifies head removal.
 *   cur walks the list; whenever cur->next->data == val,
 *   splice out cur->next.  Otherwise advance cur.
 *
 * Time : O(n).
 * Space : O(1).
 */
void removeValue(SLL *L, int val) {
    SLLNode dummy, *cur, *del;

    dummy.next = L->head;
    cur        = &dummy;

    while (cur->next != NULL) {
        if (cur->next->data == val) {
            del        = cur->next;
            cur->next  = del->next;
            if (del->next == NULL)
                L->tail = (cur == &dummy) ? NULL : cur;
            free(del);
```

```
            L->sz--;
        } else {
            cur = cur->next;
        }
    }
    L->head = dummy.next;
    if (L->head == NULL)
        L->tail = NULL;
}
```

Listing 20: Problem 10 — Remove all occurrences of val from SLL.

Problem 11 — Find the k-th Node from the End

Statement. Return the *value* of the k-th node from the end (1-indexed). Return -1 if $k > n$.

Example. $\langle 1, 2, 3, 4, 5\rangle$, $k = 2 \Rightarrow 4$.

Source: LeetCode #19 (Remove Nth Node From End); CMU 15-122.

```
/*
 * kthFromEnd : return the value of the k-th node from the end.
 *
 * Strategy (two-pointer gap):
 *   Advance 'fast' k steps ahead of 'slow'.
 *   Then advance both until fast reaches the end.
 *   'slow' is then at position n-k (0-indexed), i.e. k-th from end.
 *
 * Time : O(n) -- single pass.
 * Space : O(1).
 */
int kthFromEnd(SLL L, int k) {
    SLLNode *slow, *fast;
    int i;

    if (k < 1 || k > L.sz)
        return -1;

    fast = L.head;
    for (i = 0; i < k; i++)
        fast = fast->next;

    slow = L.head;
    while (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow->data;
}
```

Listing 21: Problem 11 — k-th from end via two-pointer gap technique.

Problem 12 — Insertion Sort on SLL

Statement. Sort an SLL in non-decreasing order using insertion sort on values.

Example. $\langle 5, 2, 8, 1, 3\rangle \Rightarrow \langle 1, 2, 3, 5, 8\rangle$.

Source: Cormen et al. CLRS 4th ed., §2.1; Sedgewick & Wayne §2.1.

```c
/*
 * insertionSort : sort L in non-decreasing order.
 *
 * Strategy: standard insertion sort on values using
 *           getItem / setItem (each O(n) for SLL).
 *
 * Time : O(n^2) -- O(n) iterations each O(n) for SLL traversal.
 * Space : O(1).
 */
void insertionSort(SLL *L) {
    int i, j, key;

    for (i = 1; i < size(*L); i++) {
        key = getItem(*L, i);
        j = i - 1;
        while (j >= 0 && getItem(*L, j) > key) {
            setItem(L, getItem(*L, j), j + 1);
            j--;
        }
        setItem(L, key, j + 1);
    }
}
```

Listing 22: Problem 12 — Insertion sort via ADT value operations.

Problem 13 — Merge Sort on SLL

Statement. Sort an SLL in non-decreasing order using merge sort (divide-and-conquer).

Constraints. $O(n \log n)$ time. Uses pointer-level split and merge.

Source: Cormen et al. CLRS 4th ed., §2.3; LeetCode #148 (Sort List).

```c
/*
 * splitHalf : split L into two halves using slow-fast pointers.
 * front receives the first half; back receives the second half.
 */
void splitHalf(SLL *L, SLL *front, SLL *back) {
    SLLNode *slow, *fast;

    if (L->head == NULL || L->head->next == NULL) {
        *front = *L;
        init(back);
        return;
    }

    slow = L->head;
    fast = L->head->next;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    /* slow is at the last node of the first half */
    front->head = L->head;
    front->tail = slow;
    back->head  = slow->next;
```

```
    back->tail  = L->tail;
    front->sz   = L->sz / 2;
    back->sz    = L->sz - front->sz;
    slow->next  = NULL;
}

/*
 * mergeSort : sort L[head..tail] using divide-and-conquer.
 * Time : O(n log n).  Space : O(log n) stack frames.
 */
void mergeSort(SLL *L) {
    SLL left, right, merged;

    if (L->sz <= 1)
        return;

    splitHalf(L, &left, &right);
    mergeSort(&left);
    mergeSort(&right);
    mergeSortedFast(&left, &right, &merged);
    *L = merged;
}
```

Listing 23: Problem 13 — Merge sort on SLL (pointer-level).

Problem 14 — Count Occurrences of a Value

Statement. Return the number of times `val` appears in L.

Example. $\langle 3,1,4,1,5,9,2,6,1\rangle$, `val`$=1 \Rightarrow 3$.

Source: Stanford CS106B; CMU 15-122 Unit 1.

```
int countOccurrences(SLL L, int val) {
    int i, count;
    count = 0;
    for (i = 0; i < size(L); i++)
        if (getItem(L, i) == val)
            count++;
    return count;
}
```

Listing 24: Problem 14 — Count occurrences of a value.

Problem 15 — Linear Search

Statement. Return the index of the first occurrence of `key`, or -1 if not found.

Source: CMU 15-122 Unit 1; Sedgewick & Wayne §1.

```
int linearSearch(SLL L, int key) {
    int i;
    for (i = 0; i < size(L); i++)
        if (getItem(L, i) == key)
            return i;
    return -1;
}
```

Listing 25: Problem 15 — Linear search on SLL.

Problem 16 — Maximum and Minimum Element

Statement. Write `maxElement` and `minElement` for a non-empty SLL.

Source: Standard CS1 exercise; Sedgewick & Wayne §1.

```
int maxElement(SLL L) {
    SLLNode *cur;
    int best;
    best = L.head->data;
    cur  = L.head->next;
    while (cur != NULL) {
        if (cur->data > best) best = cur->data;
        cur = cur->next;
    }
    return best;
}

int minElement(SLL L) {
    SLLNode *cur;
    int best;
    best = L.head->data;
    cur  = L.head->next;
    while (cur != NULL) {
        if (cur->data < best) best = cur->data;
        cur = cur->next;
    }
    return best;
}
```

Listing 26: Problem 16 — Max and min in O(n).

Problem 17 — Prefix Sum Array

Statement. Given SLL A, build a new SLL P where $P[i] = A[0] + \cdots + A[i]$.

Example. $\langle 1, 2, 3, 4, 5 \rangle \Rightarrow \langle 1, 3, 6, 10, 15 \rangle$.

Source: MIT 6.006 Lecture 1; competitive programming staple.

```
void prefixSum(SLL A, SLL *P) {
    SLLNode *cur;
    int running;

    init(P);
    running = 0;
    cur = A.head;
    while (cur != NULL) {
        running += cur->data;
        push_back(P, running);
        cur = cur->next;
    }
}
```

Listing 27: Problem 17 — Prefix sum array.

Problem 18 — Two Sum on Sorted SLL

Statement. Given a sorted SLL and a target, return 1 if two distinct indices $i < j$ exist such that $L[i] + L[j] = \text{target}$.

Source: LeetCode #167 (Two Sum II).

```
int twoSum(SLL L, int target) {
    int lo, hi, sum;
    lo = 0;
    hi = size(L) - 1;
    while (lo < hi) {
        sum = getItem(L, lo) + getItem(L, hi);
        if (sum == target) return 1;
        else if (sum < target) lo++;
        else hi--;
    }
    return 0;
}
```

Listing 28: Problem 18 — Two sum on sorted SLL.

Problem 19 — Interleave Two SLLs

Statement. Given SLLs A and B, produce C by alternating elements; append any remainder from the longer list.

Source: Stanford CS106B Programming Assignment 2.

```
void interleave(SLL A, SLL B, SLL *C) {
    int i, na, nb;
    init(C);
    na = size(A);
    nb = size(B);
    i  = 0;
    while (i < na && i < nb) {
        push_back(C, getItem(A, i));
        push_back(C, getItem(B, i));
        i++;
    }
    while (i < na) { push_back(C, getItem(A, i)); i++; }
    while (i < nb) { push_back(C, getItem(B, i)); i++; }
}
```

Listing 29: Problem 19 — Interleave two SLLs.

Problem 20 — Check if SLL is Sorted

Statement. Return 1 if L is sorted in non-decreasing order, 0 otherwise.

Source: CMU 15-122 Array Invariants lecture (adapted to SLL).

```
int isSorted(SLL L) {
    SLLNode *cur;
    cur = L.head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->data > cur->next->data)
            return 0;
        cur = cur->next;
```

```
    }
    return 1;
}
```

Listing 30: Problem 20 — Sorted check via adjacent pointer walk.

Problem 21 — Kadane's Algorithm (Maximum Subarray Sum)

Statement. Find the maximum sum of any contiguous sub-sequence of L.

Example. $\langle -2, 1, -3, 4, -1, 2, 1, -5, 4 \rangle \Rightarrow 6$.

Source: Kadane (1984); CMU 15-451; Cormen et al. CLRS §15.

```
int maxSubarraySum(SLL L) {
    SLLNode *cur;
    int current, best, x;

    if (isEmpty(L)) return 0;

    current = L.head->data;
    best    = current;
    cur     = L.head->next;

    while (cur != NULL) {
        x = cur->data;
        current = (x > current + x) ? x : current + x;
        if (current > best) best = current;
        cur = cur->next;
    }
    return best;
}
```

Listing 31: Problem 21 — Kadane's algorithm on SLL.

Problem 22 — Remove Duplicates from an Unsorted SLL

Statement. Remove all duplicate values from an unsorted SLL in-place, keeping only the first occurrence of each value.

Example. $\langle 4, 3, 2, 7, 8, 2, 3, 1 \rangle \Rightarrow \langle 4, 3, 2, 7, 8, 1 \rangle$.

Source: CMU 15-122 array exercises; Cracking the Coding Interview (6th ed.) §2.1.

```
/*
 * removeDuplicatesUnsorted :
 * For each anchor node, scan all subsequent nodes and remove
 * any whose data matches the anchor.
 *
 * Time : O(n^2) -- nested traversal.
 * Space : O(1).
 */
void removeDuplicatesUnsorted(SLL *L) {
    SLLNode *anchor, *prev, *cur, *del;

    anchor = L->head;
    while (anchor != NULL) {
        prev = anchor;
        cur  = anchor->next;
```

```
        while (cur != NULL) {
            if (cur->data == anchor->data) {
                del        = cur;
                prev->next = cur->next;
                if (cur->next == NULL)
                    L->tail = prev;
                cur = cur->next;
                free(del);
                L->sz--;
            } else {
                prev = cur;
                cur  = cur->next;
            }
        }
        anchor = anchor->next;
    }
}
```

Listing 32: Problem 22 — Remove unsorted duplicates $O(n^2)$.

Problem 23 — Intersection of Two Sorted SLLs

Statement. Given two sorted SLLs A and B, produce sorted C containing values common to both, without duplicates.

Source: LeetCode #160 (variant); MIT 6.006 sorted-list problems.

```
void intersection(SLL A, SLL B, SLL *C) {
    SLLNode *a, *b;
    int last;

    init(C);
    a    = A.head;
    b    = B.head;
    last = -1;   /* sentinel */

    while (a != NULL && b != NULL) {
        if (a->data == b->data) {
            if (a->data != last) {
                push_back(C, a->data);
                last = a->data;
            }
            a = a->next;
            b = b->next;
        } else if (a->data < b->data) {
            a = a->next;
        } else {
            b = b->next;
        }
    }
}
```

Listing 33: Problem 23 — Intersection of two sorted SLLs.

Problem 24 — Dutch National Flag (3-Way Partition)

Statement. Given an SLL containing only $\{0, 1, 2\}$, rearrange it so all 0s come first, then 1s, then 2s.

Example. $\langle 2, 0, 2, 1, 1, 0 \rangle \Rightarrow \langle 0, 0, 1, 1, 2, 2 \rangle$.

Source: Dijkstra (1976); LeetCode #75; MIT 6.006.

```
void dutchFlag(SLL *L) {
    int lo, mid, hi, tmp;

    lo  = 0;
    mid = 0;
    hi  = size(*L) - 1;

    while (mid <= hi) {
        if (getItem(*L, mid) == 0) {
            tmp = getItem(*L, lo);
            setItem(L, getItem(*L, mid), lo);
            setItem(L, tmp, mid);
            lo++;
            mid++;
        } else if (getItem(*L, mid) == 1) {
            mid++;
        } else {
            tmp = getItem(*L, hi);
            setItem(L, getItem(*L, mid), hi);
            setItem(L, tmp, mid);
            hi--;
        }
    }
}
```

Listing 34: Problem 24 — Dutch National Flag via value-level ADT.

Problem 25 — Run-Length Encoding

Statement. Encode L as alternating (value, count) pairs in C.

Example. $\langle 1, 1, 1, 2, 3, 3 \rangle \Rightarrow \langle 1, 3, 2, 1, 3, 2 \rangle$.

Source: LeetCode #443 (adapted); UC Berkeley CS61B.

```
void runLengthEncode(SLL L, SLL *C) {
    SLLNode *cur;
    int count, val;

    init(C);
    if (isEmpty(L)) return;

    cur   = L.head;
    val   = cur->data;
    count = 1;
    cur   = cur->next;

    while (cur != NULL) {
        if (cur->data == val) {
            count++;
        } else {
            push_back(C, val);
            push_back(C, count);
            val   = cur->data;
            count = 1;
```

```
        }
        cur = cur->next;
    }
    push_back(C, val);
    push_back(C, count);
}
```

Listing 35: Problem 25 — Run-length encoding on SLL.

Problem 26 — Missing Number (0 to n)

Statement. Given an SLL of n distinct integers from $\{0, \ldots, n\}$ with one missing, return the missing number.

Source: LeetCode #268; MIT 6.006 PS variant.

```
int missingNumber(SLL L) {
    int n, expected, actual;
    SLLNode *cur;

    n        = size(L);
    expected = n * (n + 1) / 2;
    actual   = 0;
    cur      = L.head;
    while (cur != NULL) {
        actual += cur->data;
        cur = cur->next;
    }
    return expected - actual;
}
```

Listing 36: Problem 26 — Missing number via Gauss formula.

Problem 27 — Boyer-Moore Majority Vote

Statement. Return the element appearing more than $\lfloor n/2 \rfloor$ times. Guaranteed to exist.

Source: Boyer-Moore (1981); LeetCode #169.

```
int majorityElement(SLL L) {
    SLLNode *cur;
    int candidate, count;

    candidate = L.head->data;
    count     = 1;
    cur       = L.head->next;

    while (cur != NULL) {
        if (count == 0) {
            candidate = cur->data;
            count     = 1;
        } else if (cur->data == candidate) {
            count++;
        } else {
            count--;
        }
        cur = cur->next;
    }
```

```
    return candidate;
}
```

Listing 37: Problem 27 — Boyer-Moore majority vote on SLL.

Problem 28 — Rotate SLL Left by k Positions

Statement. Rotate L left by k positions.

Example. $\langle 1,2,3,4,5\rangle$, $k=2 \Rightarrow \langle 3,4,5,1,2\rangle$.

Source: LeetCode #61 (Rotate List); MIT 6.006.

```
/*
 * rotateLeft : left-rotate L by k positions.
 *
 * Strategy (make-cycle):
 *   1. Traverse to find the tail; connect tail->next = head
 *      (forming a circular list).
 *   2. The new tail is at position (n - k % n - 1).
 *   3. Break the cycle there.
 *
 * Time : O(n).
 * Space : O(1).
 */
void rotateLeft(SLL *L, int k) {
    SLLNode *cur;
    int n, k_eff, steps, i;

    n = L->sz;
    if (n <= 1) return;

    k_eff = k % n;
    if (k_eff == 0) return;

    /* Connect tail to head */
    L->tail->next = L->head;

    /* New tail is at position (k_eff - 1) from the old head */
    steps = k_eff - 1;
    cur   = L->head;
    for (i = 0; i < steps; i++)
        cur = cur->next;

    /* New head is one step further */
    L->head      = cur->next;
    cur->next    = NULL;
    L->tail      = cur;
}
```

Listing 38: Problem 28 — Left rotation by k (make-cycle technique).

Problem 29 — Rotate SLL Right by k Positions

Statement. Rotate L right by k positions.

Example. $\langle 1,2,3,4,5\rangle$, $k=2 \Rightarrow \langle 4,5,1,2,3\rangle$.

Source: LeetCode #61 (Rotate List, right variant).

```c
void rotateRight(SLL *L, int k) {
    int n, k_eff;

    n = L->sz;
    if (n <= 1) return;

    k_eff = k % n;
    if (k_eff == 0) return;

    rotateLeft(L, n - k_eff);  /* right rotate k == left rotate n-k */
}
```

Listing 39: Problem 29 — Right rotation by k.

Problem 30 — Copy an SLL

Statement. Produce an independent copy of A in B.

Source: CMU 15-122 Unit 1; foundational.

```c
void copyList(SLL A, SLL *B) {
    SLLNode *cur;
    init(B);
    cur = A.head;
    while (cur != NULL) {
        push_back(B, cur->data);
        cur = cur->next;
    }
}
```

Listing 40: Problem 30 — Deep copy of SLL.

Part II: Doubly-Linked List (DLL)

6 DLL — Interface Contract

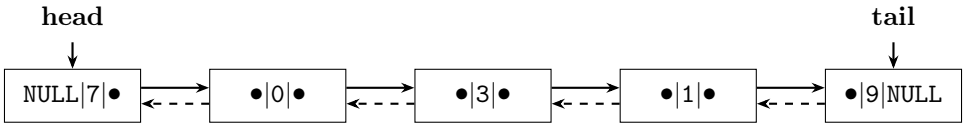
The DLL exposes the identical List ADT interface. The key improvement over the SLL is $O(1)$ `pop_back`: the `prev` pointer makes backwards traversal free.

Operation	Signature (C)	Semantics
`init`	`void init(DLL *L)`	Initialise empty list.
`push_back`	`int push_back(DLL *L, int x)`	Append x at the rear.
`push_front`	`int push_front(DLL *L, int x)`	Prepend x at the front.
`insert`	`int insert(DLL *L, int x, int pos)`	Insert x before index `pos`.
`pop_back`	`int pop_back(DLL *L, int *out)`	Remove last element into `*out`. $O(1)$.
`pop_front`	`int pop_front(DLL *L, int *out)`	Remove first element into `*out`. $O(1)$.
`removeAt`	`int removeAt(DLL *L, int pos)`	Remove element at index `pos`.
`getItem`	`int getItem(DLL L, int pos)`	Return element at index `pos`.
`setItem`	`int setItem(DLL *L, int x, int pos)`	Overwrite element at index `pos`.
`size`	`int size(DLL L)`	Return number of elements.
`isEmpty`	`int isEmpty(DLL L)`	Return 1 if empty, 0 otherwise.
`isFull`	`int isFull(DLL L)`	Always returns 0 (unbounded).
`freeList`	`void freeList(DLL *L)`	Free all nodes; reset to empty.

Table 3: DLL ADT interface contract.

7 DLL — Implementation

7.1 Memory Layout



size = 5 (solid = next, dashed = prev)

7.2 Struct Definition

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct DLLNode {
    int data;               /* payload                          */
    struct DLLNode *next;   /* pointer to the next node         */
    struct DLLNode *prev;   /* pointer to the previous node     */
} DLLNode;

typedef struct {
    DLLNode *head;          /* pointer to the first node        */
    DLLNode *tail;          /* pointer to the last node         */
    int      sz;            /* number of valid elements         */
} DLL;
```

Listing 41: DLL node and list type definitions.

7.3 Internal Node Allocator

```c
DLLNode *newDLLNode(int data) {
    DLLNode *n = (DLLNode *) malloc(sizeof(DLLNode));
    if (n == NULL)
        return NULL;
    n->data = data;
    n->next = NULL;
    n->prev = NULL;
    return n;
}
```

Listing 42: newDLLNode — allocate and initialise a DLL node.

7.4 Complete Implementation

```c
void init(DLL *L) {
    L->head = NULL;
    L->tail = NULL;
    L->sz   = 0;
}

int size(DLL L)    { return L.sz; }
int isEmpty(DLL L) { return (L.sz == 0); }
int isFull(DLL L)  { (void) L; return 0; }
```

Listing 43: DLL — init, size, isEmpty, isFull.

```c
/*
 * getItem : O(n) traversal.
 * Optimisation: start from head if pos < sz/2,
 *               start from tail otherwise.
 */
int getItem(DLL L, int pos) {
    DLLNode *cur;
    int i;

    if (pos < 0 || pos >= L.sz)
        return -1;

    if (pos <= L.sz / 2) {
        cur = L.head;
        for (i = 0; i < pos; i++)
            cur = cur->next;
    } else {
        cur = L.tail;
        for (i = L.sz - 1; i > pos; i--)
            cur = cur->prev;
    }
    return cur->data;
}

int setItem(DLL *L, int x, int pos) {
    DLLNode *cur;
    int i;

    if (pos < 0 || pos >= L->sz)
        return 0;

```

```
32      if (pos <= L->sz / 2) {
33          cur = L->head;
34          for (i = 0; i < pos; i++)
35              cur = cur->next;
36      } else {
37          cur = L->tail;
38          for (i = L->sz - 1; i > pos; i--)
39              cur = cur->prev;
40      }
41      cur->data = x;
42      return 1;
43  }
```

Listing 44: DLL — getItem, setItem.

```
1   /*
2    * push_back : O(1) -- direct tail access.
3    */
4   int push_back(DLL *L, int x) {
5       DLLNode *n = newDLLNode(x);
6       if (n == NULL) return 0;
7       if (L->tail == NULL) {
8           L->head = n;
9           L->tail = n;
10      } else {
11          n->prev        = L->tail;
12          L->tail->next  = n;
13          L->tail        = n;
14      }
15      L->sz++;
16      return 1;
17  }
18
19  /*
20   * push_front : O(1) -- direct head access.
21   */
22  int push_front(DLL *L, int x) {
23      DLLNode *n = newDLLNode(x);
24      if (n == NULL) return 0;
25      if (L->head == NULL) {
26          L->head = n;
27          L->tail = n;
28      } else {
29          n->next        = L->head;
30          L->head->prev  = n;
31          L->head        = n;
32      }
33      L->sz++;
34      return 1;
35  }
```

Listing 45: DLL — push_back, push_front.

```
1   /*
2    * insert : insert x before position pos.
3    * Time : O(n) traversal; O(1) pointer re-wiring.
4    */
5   int insert(DLL *L, int x, int pos) {
```

```
    DLLNode *n, *cur;
    int i;

    if (pos < 0 || pos > L->sz)
        return 0;
    if (pos == 0)    return push_front(L, x);
    if (pos == L->sz) return push_back(L, x);

    n = newDLLNode(x);
    if (n == NULL) return 0;

    /* Traverse to the node currently at pos */
    if (pos <= L->sz / 2) {
        cur = L->head;
        for (i = 0; i < pos; i++) cur = cur->next;
    } else {
        cur = L->tail;
        for (i = L->sz - 1; i > pos; i--) cur = cur->prev;
    }

    /* Insert n before cur */
    n->next         = cur;
    n->prev         = cur->prev;
    cur->prev->next = n;
    cur->prev       = n;
    L->sz++;
    return 1;
}
```

Listing 46: DLL — insert.

```
/*
 * pop_front : O(1) -- head pointer + prev pointer update.
 */
int pop_front(DLL *L, int *out) {
    DLLNode *old;
    if (L->head == NULL) return 0;
    old     = L->head;
    *out    = old->data;
    L->head = old->next;
    if (L->head != NULL)
        L->head->prev = NULL;
    else
        L->tail = NULL;
    free(old);
    L->sz--;
    return 1;
}

/*
 * pop_back : O(1) -- tail pointer + prev pointer.
 * KEY advantage over SLL: no traversal needed.
 */
int pop_back(DLL *L, int *out) {
    DLLNode *old;
    if (L->tail == NULL) return 0;
    old     = L->tail;
    *out    = old->data;
```

```
28      L->tail = old->prev;
29      if (L->tail != NULL)
30          L->tail->next = NULL;
31      else
32          L->head = NULL;
33      free(old);
34      L->sz--;
35      return 1;
36  }
```

Listing 47: DLL — pop_front, pop_back (both O(1)).

```
1   /*
2    * removeAt : remove element at position pos.
3    * Time : O(n) traversal; O(1) re-wiring once found.
4    */
5   int removeAt(DLL *L, int pos) {
6       DLLNode *cur;
7       int i, dummy;
8
9       if (pos < 0 || pos >= L->sz) return 0;
10      if (pos == 0)        return pop_front(L, &dummy);
11      if (pos == L->sz-1)  return pop_back(L, &dummy);
12
13      /* Traverse to pos */
14      if (pos <= L->sz / 2) {
15          cur = L->head;
16          for (i = 0; i < pos; i++) cur = cur->next;
17      } else {
18          cur = L->tail;
19          for (i = L->sz - 1; i > pos; i--) cur = cur->prev;
20      }
21
22      cur->prev->next = cur->next;
23      cur->next->prev = cur->prev;
24      free(cur);
25      L->sz--;
26      return 1;
27  }
```

Listing 48: DLL — removeAt.

```
1   void freeList(DLL *L) {
2       DLLNode *cur, *nxt;
3       cur = L->head;
4       while (cur != NULL) {
5           nxt = cur->next;
6           free(cur);
7           cur = nxt;
8       }
9       L->head = NULL;
10      L->tail = NULL;
11      L->sz   = 0;
12  }
13
14  void print(DLL L) {
15      DLLNode *cur;
16      printf("List (size=%d): <", L.sz);
```

```
17      cur = L.head;
18      while (cur != NULL) {
19          printf("%d", cur->data);
20          if (cur->next != NULL) printf(", ");
21          cur = cur->next;
22      }
23      printf(">\n");
24  }
25
26  void printReverse(DLL L) {
27      DLLNode *cur;
28      printf("Reverse (size=%d): <", L.sz);
29      cur = L.tail;
30      while (cur != NULL) {
31          printf("%d", cur->data);
32          if (cur->prev != NULL) printf(", ");
33          cur = cur->prev;
34      }
35      printf(">\n");
36  }
```

Listing 49: DLL — freeList and print utility.

7.5 Operation Complexity Summary — DLL

Operation	Time	Note
`init`, `freeList`	$O(1)$ / $O(n)$	freeList traverses all nodes
`push_back`, `push_front`	$O(1)$	Direct head/tail access
`pop_back`, `pop_front`	$O(1)$	prev pointer eliminates traversal
`insert(pos)`, `removeAt(pos)`	$O(n)$	Traverse to pos
`getItem`, `setItem`	$O(n/2)$	Bidirectional traversal from nearer end
`size`, `isEmpty`, `isFull`	$O(1)$	Field reads

Table 4: DLL time complexity summary.

7.6 SLL vs. DLL — Side-by-Side Comparison

Property	SLL (head + tail)	DLL (head + tail + prev)
Memory per node	`data + next`	`data + next + prev`
`push_back`	$O(1)$	$O(1)$
`pop_back`	$O(n)$ — must traverse	$O(1)$ — tail.prev
`pop_front`	$O(1)$	$O(1)$
`insert(pos)`	$O(n)$	$O(n)$
`removeAt(pos)`	$O(n)$	$O(n)$
`getItem`	$O(n)$	$O(n/2)$ avg
Reverse traversal	Not supported	$O(n)$ via `prev`
Deque-suitable	No ($O(n)$ rear remove)	Yes (both ends $O(1)$)

Table 5: SLL vs. DLL structural trade-offs.

8 DLL — Practice Problems

Problem 31 — Reverse a DLL In-Place

Statement. Reverse L in-place by swapping `next` and `prev` pointers for every node, then updating `head` and `tail`.

Example. $\langle 1, 2, 3, 4, 5\rangle \Rightarrow \langle 5, 4, 3, 2, 1\rangle$.

Source: LeetCode #206 (adapted); CMU 15-122.

```
/*
 * reverse : swap next and prev on every node; exchange head and tail.
 *
 * Time : O(n) -- single pass.
 * Space : O(1).
 */
void reverse(DLL *L) {
    DLLNode *cur, *tmp;

    cur = L->head;
    while (cur != NULL) {
        tmp       = cur->next;
        cur->next = cur->prev;
        cur->prev = tmp;
        cur       = tmp;        /* advance using saved 'next' */
    }
    /* Swap head and tail */
    tmp     = L->head;
    L->head = L->tail;
    L->tail = tmp;
}
```

Listing 50: Problem 31 — Reverse DLL by swapping next/prev.

Problem 32 — Deque (Double-Ended Queue) Built on DLL

Statement. Implement a deque supporting $O(1)$ insertion and deletion at both ends. All four operations are $O(1)$ because the DLL has direct head and tail access with `prev` pointers.

Source: CMU 15-122 Unit 2; MIT 6.006; Sedgewick & Wayne §1.3.

```
typedef DLL Deque;

void dInit(Deque *D)  { init(D); }
int  dIsEmpty(Deque D){ return isEmpty(D); }
int  dSize(Deque D)   { return size(D); }
void dFree(Deque *D)  { freeList(D); }

/* addFront : O(1). */
int addFront(Deque *D, int x) { return push_front(D, x); }

/* addRear : O(1). */
int addRear(Deque *D, int x)  { return push_back(D, x); }

/* removeFront : O(1). */
int removeFront(Deque *D, int *out) {
    if (isEmpty(*D)) return 0;
    return pop_front(D, out);
}

/* removeRear : O(1). */
int removeRear(Deque *D, int *out) {
    if (isEmpty(*D)) return 0;
    return pop_back(D, out);
}

/* peekFront : return front value without removing. O(1). */
int peekFront(Deque D, int *out) {
    if (isEmpty(D)) return 0;
    *out = D.head->data;
    return 1;
}

/* peekRear : return rear value without removing. O(1). */
int peekRear(Deque D, int *out) {
    if (isEmpty(D)) return 0;
    *out = D.tail->data;
    return 1;
}
```

Listing 51: Problem 32 — Deque built on DLL (all O(1) endpoints).

Problem 33 — Stack ADT Built on DLL

Statement. Implement a stack using a DLL. Push/pop at the `tail` for $O(1)$.

Source: CMU 15-122 Unit 2; Sedgewick & Wayne §1.3.

```
typedef DLL Stack;

void sInit(Stack *S)  { init(S); }
int  sIsEmpty(Stack S){ return isEmpty(S); }
void sFree(Stack *S)  { freeList(S); }
```

```
6
7  int sPush(Stack *S, int x) { return push_back(S, x); }
8
9  int sPop(Stack *S, int *out) {
10     if (isEmpty(*S)) return 0;
11     return pop_back(S, out);   /* O(1) because DLL has prev */
12 }
13
14 int sPeek(Stack S, int *out) {
15     if (isEmpty(S)) return 0;
16     *out = S.tail->data;
17     return 1;
18 }
```

Listing 52: Problem 33 — Stack built on DLL.

Problem 34 — Remove Duplicates from Sorted DLL

Statement. Given a sorted DLL, remove all duplicate values so each appears exactly once.

Source: LeetCode #82, #83; CMU 15-122.

```
1  void removeDuplicatesSorted(DLL *L) {
2      DLLNode *cur, *dup;
3
4      cur = L->head;
5      while (cur != NULL && cur->next != NULL) {
6          if (cur->data == cur->next->data) {
7              dup = cur->next;
8              cur->next = dup->next;
9              if (dup->next != NULL)
10                 dup->next->prev = cur;
11             else
12                 L->tail = cur;
13             free(dup);
14             L->sz--;
15         } else {
16             cur = cur->next;
17         }
18     }
19 }
```

Listing 53: Problem 34 — Remove adjacent duplicates from sorted DLL.

Problem 35 — Merge Two Sorted DLLs

Statement. Given sorted DLLs A and B, produce sorted C.

Source: Cormen et al. CLRS 4th ed., §2.3; LeetCode #21 adapted.

```
1  void mergeSorted(DLL A, DLL B, DLL *C) {
2      int ia, ib;
3      init(C);
4      ia = 0; ib = 0;
5      while (ia < size(A) && ib < size(B)) {
6          if (getItem(A, ia) <= getItem(B, ib)) {
7              push_back(C, getItem(A, ia++));
8          } else {
9              push_back(C, getItem(B, ib++));
```

```
        }
    }
    while (ia < size(A)) push_back(C, getItem(A, ia++));
    while (ib < size(B)) push_back(C, getItem(B, ib++));
}
```

Listing 54: Problem 35 — Merge two sorted DLLs (O(n+m) pointer level).

Problem 36 — Merge Sort on DLL

Statement. Sort a DLL using merge sort.

Constraints. $O(n \log n)$ time.

Source: Cormen et al. CLRS 4th ed., §2.3; LeetCode #148.

```
/* splitHalf : split L into front (first half) and back (second half).
   */
void splitHalfDLL(DLL *L, DLL *front, DLL *back) {
    DLLNode *slow, *fast;
    int half;

    if (L->sz <= 1) {
        *front = *L;
        init(back);
        return;
    }

    slow = L->head;
    fast = L->head->next;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    /* slow is the last node of the front half */
    front->head = L->head;
    front->tail = slow;
    back->head  = slow->next;
    back->tail  = L->tail;
    half        = L->sz / 2;
    front->sz   = half;
    back->sz    = L->sz - half;

    slow->next->prev = NULL;
    slow->next       = NULL;
    front->head->prev = NULL;
}

void mergeSort(DLL *L) {
    DLL left, right, merged;

    if (L->sz <= 1) return;

    splitHalfDLL(L, &left, &right);
    mergeSort(&left);
    mergeSort(&right);
    mergeSorted(left, right, &merged);
```

```
    freeList(&left);
    freeList(&right);
    *L = merged;
}
```

Listing 55: Problem 36 — Merge sort on DLL.

Problem 37 — Palindrome Check on DLL

Statement. Return 1 if L reads the same forwards and backwards. Exploit the `prev` pointer for $O(n)$ bidirectional traversal.

Source: LeetCode #234; Stanford CS106B.

```
/*
 * isPalindrome : use both head (lo) and tail (hi) pointers.
 * Advance lo forward; retreat hi backward; compare data.
 *
 * Time : O(n).
 * Space : O(1).
 */
int isPalindrome(DLL L) {
    DLLNode *lo, *hi;

    lo = L.head;
    hi = L.tail;

    while (lo != hi && lo->prev != hi) {
        if (lo->data != hi->data)
            return 0;
        lo = lo->next;
        hi = hi->prev;
    }
    /* Check the middle pair (or middle element) */
    if (lo != hi && lo->data != hi->data)
        return 0;
    return 1;
}
```

Listing 56: Problem 37 — Palindrome check using DLL head and tail.

Problem 38 — Move All Zeros to the End (DLL)

Statement. Move all zero elements to the end of L preserving relative order.

Source: LeetCode #283.

```
void moveZeros(DLL *L) {
    int w, i, n;
    n = size(*L);
    w = 0;
    for (i = 0; i < n; i++) {
        if (getItem(*L, i) != 0) {
            setItem(L, getItem(*L, i), w);
            w++;
        }
    }
    for (i = w; i < n; i++)
```

```
        setItem(L, 0, i);
}
```

Listing 57: Problem 38 — Move zeros to end on DLL (write-pointer).

Problem 39 — Remove All Occurrences of a Value (DLL)

Statement. Remove every node whose `data == val` from L in-place.

Source: LeetCode #203 adapted; CMU 15-122.

```
void removeValue(DLL *L, int val) {
    DLLNode *cur, *del;

    cur = L->head;
    while (cur != NULL) {
        if (cur->data == val) {
            del = cur;
            cur = cur->next;
            /* Update neighbours */
            if (del->prev != NULL)
                del->prev->next = del->next;
            else
                L->head = del->next;
            if (del->next != NULL)
                del->next->prev = del->prev;
            else
                L->tail = del->prev;
            free(del);
            L->sz--;
        } else {
            cur = cur->next;
        }
    }
}
```

Listing 58: Problem 39 — Remove all occurrences of val from DLL.

Problem 40 — Rotate DLL Left by k Positions

Statement. Left-rotate L by k positions.

Source: LeetCode #61; MIT 6.006.

```
void rotateLeft(DLL *L, int k) {
    int n, k_eff, i;
    DLLNode *new_tail;

    n = L->sz;
    if (n <= 1) return;
    k_eff = k % n;
    if (k_eff == 0) return;

    /* Connect tail to head to form a cycle */
    L->tail->next  = L->head;
    L->head->prev  = L->tail;

    /* New tail is at position k_eff - 1 from old head */
    new_tail = L->head;
```

```
    for (i = 0; i < k_eff - 1; i++)
        new_tail = new_tail->next;

    L->head           = new_tail->next;
    L->head->prev     = NULL;
    new_tail->next    = NULL;
    L->tail           = new_tail;
}
```

Listing 59: Problem 40 — Left rotation of DLL by k (make-cycle).

Problem 41 — Rotate DLL Right by k Positions

Source: LeetCode #61 (right variant).

```
void rotateRight(DLL *L, int k) {
    int n, k_eff;
    n = L->sz;
    if (n <= 1) return;
    k_eff = k % n;
    if (k_eff == 0) return;
    rotateLeft(L, n - k_eff);
}
```

Listing 60: Problem 41 — Right rotation of DLL.

Problem 42 — Dutch National Flag (DLL)

Statement. Given a DLL containing only $\{0, 1, 2\}$, rearrange so all 0s come first, then 1s, then 2s. Single pass.

Source: Dijkstra (1976); LeetCode #75.

```
void dutchFlag(DLL *L) {
    int lo, mid, hi, tmp;
    lo = 0; mid = 0; hi = size(*L) - 1;
    while (mid <= hi) {
        if (getItem(*L, mid) == 0) {
            tmp = getItem(*L, lo);
            setItem(L, getItem(*L, mid), lo);
            setItem(L, tmp, mid);
            lo++; mid++;
        } else if (getItem(*L, mid) == 1) {
            mid++;
        } else {
            tmp = getItem(*L, hi);
            setItem(L, getItem(*L, mid), hi);
            setItem(L, tmp, mid);
            hi--;
        }
    }
}
```

Listing 61: Problem 42 — Dutch National Flag on DLL.

Problem 43 — Run-Length Encoding (DLL)

Statement. Encode L as alternating (value, count) pairs in C.

Source: LeetCode #443 adapted; UC Berkeley CS61B.

```
void runLengthEncode(DLL L, DLL *C) {
    DLLNode *cur;
    int count, val;
    init(C);
    if (isEmpty(L)) return;
    val   = L.head->data;
    count = 1;
    cur   = L.head->next;
    while (cur != NULL) {
        if (cur->data == val) {
            count++;
        } else {
            push_back(C, val);
            push_back(C, count);
            val   = cur->data;
            count = 1;
        }
        cur = cur->next;
    }
    push_back(C, val);
    push_back(C, count);
}
```

Listing 62: Problem 43 — Run-length encoding on DLL.

Problem 44 — Prefix Sum Array (DLL)

Statement. Build DLL P where $P[i] = A[0] + \cdots + A[i]$.

Source: MIT 6.006 Lecture 1.

```
void prefixSum(DLL A, DLL *P) {
    DLLNode *cur;
    int running;
    init(P);
    running = 0;
    cur = A.head;
    while (cur != NULL) {
        running += cur->data;
        push_back(P, running);
        cur = cur->next;
    }
}

int rangeSum(DLL P, int lo, int hi) {
    if (lo == 0) return getItem(P, hi);
    return getItem(P, hi) - getItem(P, lo - 1);
}
```

Listing 63: Problem 44 — Prefix sum on DLL.

Problem 45 — Trapping Rain Water (DLL)

Statement. Given heights L, compute total units of trapped water.

Source: LeetCode #42; MIT 6.006 interview-style.

```
int trapWater(DLL L) {
    DLLNode *lo_ptr, *hi_ptr;
    int leftMax, rightMax, water;

    lo_ptr   = L.head;
    hi_ptr   = L.tail;
    leftMax  = 0;
    rightMax = 0;
    water    = 0;

    while (lo_ptr != hi_ptr && lo_ptr->prev != hi_ptr) {
        if (lo_ptr->data <= hi_ptr->data) {
            if (lo_ptr->data >= leftMax)
                leftMax = lo_ptr->data;
            else
                water += leftMax - lo_ptr->data;
            lo_ptr = lo_ptr->next;
        } else {
            if (hi_ptr->data >= rightMax)
                rightMax = hi_ptr->data;
            else
                water += rightMax - hi_ptr->data;
            hi_ptr = hi_ptr->prev;
        }
    }
    return water;
}
```

Listing 64: Problem 45 — Trapping rain water on DLL (two-pointer O(1) space).

Problem 46 — Sliding Window Maximum (DLL)

Statement. Given L and window size k, produce M where $M[i] = \max(L[i..i+k-1])$.

Source: LeetCode #239; UC Berkeley CS61B.

```
void slidingWindowMax(DLL L, int k, DLL *M) {
    int i, j, winMax, n;
    n = size(L);
    init(M);
    for (i = 0; i <= n - k; i++) {
        winMax = getItem(L, i);
        for (j = i + 1; j < i + k; j++)
            if (getItem(L, j) > winMax)
                winMax = getItem(L, j);
        push_back(M, winMax);
    }
}
```

Listing 65: Problem 46 — Sliding window maximum O(nk).

Problem 47 — Count Pairs Summing to Target (DLL)

Statement. Count all pairs (i, j) with $i < j$ such that $L[i] + L[j] = \text{target}$.

Example. $\langle 1, 5, 7, -1, 5 \rangle$, target$= 6 \Rightarrow 3$.

Source: LeetCode #1 counting variant.

```c
int countPairs(DLL L, int target) {
    int i, j, count;
    count = 0;
    for (i = 0; i < size(L) - 1; i++)
        for (j = i + 1; j < size(L); j++)
            if (getItem(L, i) + getItem(L, j) == target)
                count++;
    return count;
}
```

Listing 66: Problem 47 — Count pairs summing to target (brute-force).

Problem 48 — Subarray Sum Equal to k (DLL)

Statement. Count contiguous sub-sequences of L whose sum equals k.

Example. $\langle 1, 1, 1 \rangle$, $k = 2 \Rightarrow 2$.

Source: LeetCode #560; MIT 6.006 prefix-sum application.

```c
int subarraySum(DLL L, int k) {
    int i, j, n, count, s;
    n = size(L);
    count = 0;
    for (i = 0; i < n; i++) {
        s = 0;
        for (j = i; j < n; j++) {
            s += getItem(L, j);
            if (s == k) count++;
        }
    }
    return count;
}
```

Listing 67: Problem 48 — Subarray sum == k (prefix-sum $O(n^2)$).

Problem 49 — Remove Adjacent Duplicates (Stack-Based, DLL)

Statement. Repeatedly cancel adjacent equal pairs until none remain.

Example. $\langle 1, 2, 2, 3, 3, 3, 1 \rangle \Rightarrow \langle 1 \rangle$.

Source: LeetCode #1047; CMU 15-122.

```c
void removeAdjacentDups(DLL *L) {
    DLL S;
    DLLNode *cur;
    int x, dummy;

    init(&S);
    cur = L->head;

    while (cur != NULL) {
        x = cur->data;
        if (!isEmpty(S) && S.tail->data == x)
            pop_back(&S, &dummy);
        else
            push_back(&S, x);
```

```
        cur = cur->next;
    }

    freeList(L);
    init(L);
    cur = S.head;
    while (cur != NULL) {
        push_back(L, cur->data);
        cur = cur->next;
    }
    freeList(&S);
}
```

Listing 68: Problem 49 — Remove all adjacent duplicates (DLL stack).

Problem 50 — Segregate Even and Odd Numbers (DLL)

Statement. Rearrange L so all even numbers appear before all odd numbers. Relative order within groups need not be preserved.

Source: GeeksForGeeks classic; two-pointer pattern.

```
void segregateEvenOdd(DLL *L) {
    DLLNode *lo, *hi, tmp_val;
    int tmp;

    lo = L->head;
    hi = L->tail;

    while (lo != hi && lo->prev != hi) {
        while (lo != hi && lo->data % 2 == 0)
            lo = lo->next;
        while (lo != hi && hi->data % 2 != 0)
            hi = hi->prev;
        if (lo != hi && lo->prev != hi) {
            tmp       = lo->data;
            lo->data  = hi->data;
            hi->data  = tmp;
            lo = lo->next;
            hi = hi->prev;
        }
    }
}
```

Listing 69: Problem 50 — Segregate even and odd (DLL two-pointer).

Problem 51 — Copy and Concatenate (DLL)

Statement. (a) Produce an independent deep copy of A in B. (b) Produce $C = A$ followed by B.

Source: CMU 15-122 Unit 1; foundational.

```
void copyList(DLL A, DLL *B) {
    DLLNode *cur;
    init(B);
    cur = A.head;
    while (cur != NULL) {
```

```
        push_back(B, cur->data);
        cur = cur->next;
    }
}

void concatenate(DLL A, DLL B, DLL *C) {
    DLLNode *cur;
    init(C);
    cur = A.head;
    while (cur != NULL) { push_back(C, cur->data); cur = cur->next; }
    cur = B.head;
    while (cur != NULL) { push_back(C, cur->data); cur = cur->next; }
}
```

Listing 70: Problem 51 — Copy and concatenate DLLs.

Problem 52 — LRU Cache Eviction (DLL + Conceptual)

Statement. Design an LRU (Least Recently Used) cache of capacity c using a DLL to track access order. Implement `lruAccess(key)` which moves `key` to the front (most recently used) and evicts the tail (least recently used) when capacity is exceeded. For simplicity, keys are integers and no separate hash table is used; search is $O(n)$.

Constraints. Uses direct DLL pointer manipulation. Access: $O(n)$ without hash table. Full $O(1)$ requires a hash map (not in scope here).

Source: LeetCode #146 (LRU Cache); CMU 15-445 (Database Systems); Silberschatz *Operating System Concepts* §9 (Page Replacement).

```
typedef struct {
    DLL   order;    /* front = MRU, back = LRU */
    int   capacity;
} LRUCache;

/* lruInit : initialise cache with given capacity. */
void lruInit(LRUCache *C, int capacity) {
    init(&C->order);
    C->capacity = capacity;
}

/*
 * lruAccess : access 'key'.
 *   If key exists: remove it and re-insert at front (MRU).
 *   If key is new: insert at front; evict tail if over capacity.
 *
 * Time : O(n) -- linear scan replaces hash lookup here.
 */
void lruAccess(LRUCache *C, int key) {
    DLLNode *cur, *del;
    int dummy;

    /* Search for key */
    cur = C->order.head;
    while (cur != NULL) {
        if (cur->data == key) {
            /* Remove from current position */
            if (cur->prev != NULL)
                cur->prev->next = cur->next;
```

```
            else
                C->order.head = cur->next;
            if (cur->next != NULL)
                cur->next->prev = cur->prev;
            else
                C->order.tail = cur->prev;
            free(cur);
            C->order.sz--;
            break;
        }
        cur = cur->next;
    }
    /* Insert at front */
    push_front(&C->order, key);

    /* Evict LRU if over capacity */
    if (C->order.sz > C->capacity)
        pop_back(&C->order, &dummy);
}

void lruFree(LRUCache *C) { freeList(&C->order); }
```

Listing 71: Problem 52 — LRU cache simulation on DLL.

Problem 53 — Insertion Sort on DLL

Statement. Sort a DLL in non-decreasing order using insertion sort.

Source: Cormen et al. CLRS 4th ed., §2.1; Sedgewick & Wayne §2.1.

```
/*
 * insertionSortDLL : sort L using insertion sort on values.
 *
 * Strategy : for each node from the second onward,
 *   extract its value and find the correct insertion point
 *   in the sorted prefix via backward traversal using prev.
 *
 * Time : O(n^2) comparisons; O(1) space.
 */
void insertionSortDLL(DLL *L) {
    DLLNode *cur, *sorted;
    int key;

    if (L->sz <= 1) return;

    cur = L->head->next;
    while (cur != NULL) {
        key    = cur->data;
        sorted = cur->prev;
        while (sorted != NULL && sorted->data > key) {
            sorted->next->data = sorted->data;
            sorted = sorted->prev;
        }
        if (sorted == NULL)
            L->head->data = key;
        else
            sorted->next->data = key;
        cur = cur->next;
```

```
    }
}
```

Listing 72: Problem 53 — Insertion sort on DLL (pointer re-wiring).

Problem 54 — Flatten a Multi-Level DLL

Statement. In this extended problem each node may have a `child` pointer to a sub-list. Flatten the multi-level DLL into a single-level DLL in depth-first order.

Constraints. $O(n)$ time. Direct pointer manipulation required.

Source: LeetCode #430 (Flatten a Multilevel Doubly Linked List); CMU 15-122 advanced.

```
/*
 * Extended node with an optional child sub-list.
 * Used only for this problem; does not change the base DLL struct.
 */
typedef struct MLNode {
    int          data;
    struct MLNode *next;
    struct MLNode *prev;
    struct MLNode *child;  /* NULL for most nodes */
} MLNode;

/*
 * flattenML : flatten the multi-level list rooted at head in-place.
 * Returns the tail of the flattened segment.
 *
 * Strategy (iterative, O(n)):
 *   Walk the list. When a node with a child is found:
 *     1. Recursively flatten the child list.
 *     2. Insert the flattened child between cur and cur->next.
 *     3. Clear cur->child.
 */
MLNode *flattenML(MLNode *head) {
    MLNode *cur, *child_tail, *next_node;

    if (head == NULL) return NULL;

    cur = head;
    while (cur != NULL) {
        if (cur->child != NULL) {
            child_tail = flattenML(cur->child);
            next_node  = cur->next;

            /* Link cur -> child */
            cur->next        = cur->child;
            cur->child->prev = cur;
            cur->child       = NULL;

            /* Link child_tail -> next_node */
            child_tail->next = next_node;
            if (next_node != NULL)
                next_node->prev = child_tail;
        }
        if (cur->next == NULL)
            return cur;  /* reached true tail */
```

```
45          cur = cur->next;
46      }
47      return cur;
48  }
```

Listing 73: Problem 54 — Flatten a multi-level DLL (depth-first).

Problem 55 — Check Equality of Two DLLs

Statement. Return 1 if A and B are element-wise equal, 0 otherwise.

Source: CMU 15-122 foundational; standard CS1.

```
1  int listsEqual(DLL A, DLL B) {
2      DLLNode *a, *b;
3      if (A.sz != B.sz) return 0;
4      a = A.head;
5      b = B.head;
6      while (a != NULL) {
7          if (a->data != b->data) return 0;
8          a = a->next;
9          b = b->next;
10     }
11     return 1;
12 }
```

Listing 74: Problem 55 — Element-wise equality of two DLLs.

Problem 56 — Maximum and Minimum (DLL)

Source: Standard CS1; Sedgewick & Wayne §1.

```
1  int maxElement(DLL L) {
2      DLLNode *cur;
3      int best;
4      best = L.head->data;
5      cur  = L.head->next;
6      while (cur != NULL) {
7          if (cur->data > best) best = cur->data;
8          cur = cur->next;
9      }
10     return best;
11 }
12
13 int minElement(DLL L) {
14     DLLNode *cur;
15     int best;
16     best = L.head->data;
17     cur  = L.head->next;
18     while (cur != NULL) {
19         if (cur->data < best) best = cur->data;
20         cur = cur->next;
21     }
22     return best;
23 }
```

Listing 75: Problem 56 — Max and min on DLL.

Problem 57 — Missing Number (DLL)

Source: LeetCode #268.

```
int missingNumber(DLL L) {
    DLLNode *cur;
    int n, expected, actual;
    n        = size(L);
    expected = n * (n + 1) / 2;
    actual   = 0;
    cur      = L.head;
    while (cur != NULL) { actual += cur->data; cur = cur->next; }
    return expected - actual;
}
```

Listing 76: Problem 57 — Missing number via Gauss formula on DLL.

Problem 58 — Boyer-Moore Majority Vote (DLL)

Source: Boyer-Moore (1981); LeetCode #169.

```
int majorityElement(DLL L) {
    DLLNode *cur;
    int candidate, count;
    candidate = L.head->data;
    count     = 1;
    cur       = L.head->next;
    while (cur != NULL) {
        if (count == 0)      { candidate = cur->data; count = 1; }
        else if (cur->data == candidate) count++;
        else                 count--;
        cur = cur->next;
    }
    return candidate;
}
```

Listing 77: Problem 58 — Boyer-Moore majority vote on DLL.

Problem 59 — Sublist Search (DLL)

Statement. Return 1 if pattern P appears as a contiguous sublist in L.

Source: Naive string matching adapted; Stanford CS106B.

```
int containsSublist(DLL L, DLL P) {
    int i, j, m, n, match;
    n = size(L);
    m = size(P);
    if (m == 0) return 1;
    if (m > n)  return 0;
    for (i = 0; i <= n - m; i++) {
        match = 1;
        for (j = 0; j < m; j++) {
            if (getItem(L, i + j) != getItem(P, j)) {
                match = 0;
                break;
            }
        }
        if (match) return 1;
```

```
    }
    return 0;
}
```

Listing 78: Problem 59 — Naive sublist search on DLL.

Problem 60 — Deep Copy vs. Shallow Copy (DLL)

Statement. Implement (a) `shallowCopy` (copies struct fields only — aliased, unsafe) and (b) `deepCopy` (allocates independent nodes). Demonstrate the dangling-pointer hazard of shallow copy.

Source: CMU 15-122 Unit 5; Stroustrup *The C++ Programming Language* §7.7 (adapted to C).

```
/*
 * shallowCopy : copy struct fields only.
 * DANGER: dst.head/tail point into the SAME nodes as src.
 * Freeing src invalidates dst -- use ONLY for demonstration.
 */
DLL shallowCopy(DLL src) {
    DLL dst;
    dst.head = src.head;
    dst.tail = src.tail;
    dst.sz   = src.sz;
    return dst;
}

/*
 * deepCopy : allocate fresh nodes and copy all data.
 * dst is fully independent. Caller must call freeList(&dst).
 */
DLL deepCopy(DLL src) {
    DLL dst;
    DLLNode *cur;
    init(&dst);
    cur = src.head;
    while (cur != NULL) {
        push_back(&dst, cur->data);
        cur = cur->next;
    }
    return dst;
}

/*
 * Demonstration (run in main):
 *
 *   DLL A, B_shallow, B_deep;
 *   init(&A);
 *   push_back(&A, 1); push_back(&A, 2); push_back(&A, 3);
 *
 *   B_shallow = shallowCopy(A);
 *   B_deep    = deepCopy(A);
 *
 *   freeList(&A);
 *   // B_shallow.head is now a dangling pointer -- undefined!
 *   // B_deep is still valid: {1, 2, 3}
 *
 *   print(B_deep);   // <1, 2, 3>
```

```
45  *    freeList(&B_deep);
46  */
```

Listing 79: Problem 60 — Shallow copy (aliased) vs. deep copy (independent).

9 Known Limitations and Design Trade-offs

Limitation	Root Cause and Mitigation
Expensive getItem / setItem	Both SLL and DLL require $O(n)$ sequential traversal for random access by index. Fix: use an array-based structure (ArrayList / VectorList) when random access dominates.
SLL pop_back is $O(n)$	No `prev` pointer means the second-to-last node must be located by traversal. Fix: use a DLL where `pop_back` is $O(1)$.
Extra memory per node	Each SLL node uses one pointer; each DLL node uses two. For n integers this is $n \cdot$ `sizeof(int *)` (SLL) or $2n \cdot$ `sizeof(int *)` (DLL) of pointer overhead beyond the data. Fix: array-based structures eliminate per-node pointer overhead.
Cache performance	Linked-list nodes are scattered across the heap; array-based structures are contiguous in memory and exploit spatial locality. Modern CPUs can process array elements up to 10× faster due to cache-line prefetching.
No random access in $O(1)$	Unlike arrays, there is no direct index calculation. Indexing always requires traversal. Use hash tables or balanced BSTs when $O(1)$ keyed access is required.
Memory leaks in C	Every `init` must be matched with a `freeList`. Temporary lists inside functions (e.g., `mergeSort`) must also be freed. Verify with `valgrind -leak-check=full`.
Pointer invalidation	After a node is freed (e.g., via `removeAt` or `pop_back`), all external pointers to that node are dangling. Never cache raw node pointers across mutating operations.

10 Quick Reference Cards

SLL (head + tail)

Operation	Time	Note
`push_front`	$O(1)$	Head prepend
`push_back`	$O(1)$	Tail append
`pop_front`	$O(1)$	Head remove
`pop_back`	$O(n)$	Must traverse to second-to-last
`insert(pos)`	$O(n)$	Traverse to pos−1
`removeAt(pos)`	$O(n)$	Traverse to pos−1
`getItem`	$O(n)$	Sequential traversal
`setItem`	$O(n)$	Sequential traversal
`size, isEmpty, isFull`	$O(1)$	Field read
`init`	$O(1)$	Set fields to NULL/0
`freeList`	$O(n)$	Free every node
Stack (push/pop front)	$O(1)$	Head acts as top
Queue (enqueue/dequeue)	$O(1)$	Tail enqueue, head dequeue

Table 7: SLL quick reference.

DLL (head + tail + prev)

Operation	Time	Note
`push_front`	$O(1)$	Head prepend
`push_back`	$O(1)$	Tail append
`pop_front`	$O(1)$	Head remove + prev update
`pop_back`	$O(1)$	Tail remove via prev
`insert(pos)`	$O(n)$	Bidirectional traversal
`removeAt(pos)`	$O(n)$	Bidirectional traversal
`getItem`	$O(n/2)$ avg	Traverse from nearer end
`setItem`	$O(n/2)$ avg	Traverse from nearer end
`size, isEmpty, isFull`	$O(1)$	Field read
`init`	$O(1)$	Set fields to NULL/0
`freeList`	$O(n)$	Free every node
Stack (push/pop tail)	$O(1)$	Tail is top
Queue enqueue/dequeue	$O(1)$	Tail enqueue, head dequeue
Deque (all four ends)	$O(1)$	Full $O(1)$ deque

Table 8: DLL quick reference.

© Juevesano, D. All Rights Reserved, UPC. This document is intended solely for personal academic study and is not for redistribution or reproduction without explicit written permission.